Add-On Feature — Automated Versioning & Snapshot System

Purpose

Add a fully automated engineering version control system to the platform so that every change in the automation workflow is:

• versioned

• snapshot-able

• rollback-able

• traceable

This allows engineers to treat automation systems like software systems.

The system will automatically version:

- plant models
- PLC logic
- simulation states
- configuration files
- monitoring configuration
- engineering documents

No manual Git interaction required.

Where This Integrates in the Existing Workflow

This feature expands Step 7 — Control Logic Generation.

It adds logic lifecycle management.

Updated step behavior:

plant graph
↓
control strategy
↓
PLC logic generation
↓
automatic version commit
↓
system snapshot creation

Versioning is triggered automatically during:

• logic generation

• plant graph updates

• simulation validation

• PLC deployment

• configuration changes

Tools (All Open Source)

Version Control Engine

Git

Core version control engine.

Tracks:

- PLC logic
- configuration
- plant graph exports

Git Server

Gitea

Self-hosted lightweight Git service.

Provides:

- repository browser
- commit history
- branch visualization
- access control

Runs as a Docker container.

Versioned Engineering Database

Dolt

Git-like versioning for databases.

Used to version:

- plant graph exports
- engineering metadata

- tag lists
- configuration tables

Allows engineers to compare database states.

Python Git Interface

GitPython

Used by backend to automate commits.

Example:

```
repo.index.add(files)
repo.index.commit("Auto-generated PLC logic update")
```

All commits happen automatically.

Embedded Code Editor

Monaco Editor

Used for viewing generated PLC code.

Same editor used by VS Code.

Supports:

- syntax highlighting
- diff view
- code comparison

Optional Developer Integration

VS Code Server

Optional embedded code workspace.

Allows advanced engineers to:

- inspect generated logic
- review commits
- manually edit templates

Backend Architecture

The backend adds a Version Manager Service.

Runs inside the FastAPI backend.

Structure:

FastAPI
↓
Version Manager
↓
Git Repository
↓
Dolt Database

Versioning Workflow

1 — Automatic Commit on Logic Generation

Whenever Step 7 generates PLC logic:

rules engine
↓

ST logic templates
↓
generated PLC code

The system automatically performs:

```
git add .
git commit
```

Commit message example:

```
Auto Commit
Control Logic Generated

Plant Model Version: v1.3
Simulation Status: Validated
Timestamp: 14:22:11
```

---

2 — System Snapshot Creation

Every commit includes a full system snapshot.

Snapshot contains:

• PLC logic files

• plant graph export

• simulation model

• control rule definitions

• runtime configuration

Snapshots stored in Git.

---

3 — Plant Model Versioning

The Neo4j plant graph is exported into a versioned dataset.

Process:

Neo4j graph

↓
export JSON
↓
commit to Git

Database metadata stored in Dolt.

This allows engineers to compare plant models across versions.

Example change:

v1.1
Added Pump-102

v1.2
Added Valve-201
Updated LT-101 connection

---

4 — Deployment Version Tags

When PLC runtime is deployed:

The system creates a version tag.

Example:

git tag deployed_v1.3

This marks the exact logic version running in production.

---

5 — Automatic Version Triggers

Commits automatically occur when:

• plant graph changes

• PLC logic regenerated

• simulation passes validation

• PLC runtime deployed

• monitoring configuration updated

No user interaction required.

UI Integration

The UI follows your existing 3-panel architecture.

Top Toolbar
Left Navigator
Main Workspace
Right Engineering Panel

We add two UI components.

UI Component 1 — Version History Panel

Location

Right Engineering Panel.

New tab added:

Details
Signals
Trace
Replay
Versions
Diagnostics

Version History Display

Example:

VERSION HISTORY
———————————

v1.4

Simulation Fix
12:14 PM

v1.3
Control Logic Generated
11:48 AM

v1.2
Plant Graph Updated
10:22 AM

Clicking a version loads that state.

UI Component 2 — Snapshot Manager

Location

Top Command Bar.

New button:

Upload | Parse | Generate | Simulate | Deploy | Monitor | Replay | Versions

Clicking Versions opens snapshot manager.

Snapshot Manager Layout

SYSTEM SNAPSHOTS

[Create Snapshot]

Snapshot List
———————————

Snapshot 12
Before Simulation

Snapshot 11
After Logic Generation

Snapshot 10
Plant Model Imported

Snapshot Actions

Each snapshot has buttons:

Load Snapshot
Rollback
Compare
Export

UI Component 3 — Logic Diff Viewer

Location

Inside Code Panel.

When comparing versions.

Example display:

VERSION 1.3

```
IF LT101 > 90 THEN
PUMP101 := FALSE;
END_IF
```

VERSION 1.4

```
IF LT101 > 95 THEN
PUMP101 := FALSE;
END_IF
```

Differences highlighted.

Settings

Add new settings group.

Settings → Version Control

Enable Auto Versioning: ON
Auto Snapshot on Deploy: ON
Max Snapshots Stored: 500
Enable Database Versioning: ON
Git Repository Location: /projects/repo
Enable Git Export: ON
Enable VSCode Integration: OFF
Snapshot Retention Days: 30

Integration with Existing Features

Step 3 — Plant Graph

Graph exports versioned.

Step 5 — Simulation

Simulation models versioned.

Step 8 — PLC Runtime

Deployment version tagged.

Step 10 — Monitoring

Monitoring configuration included in snapshots.

Step 10.5 — Replay

Replay data linked to snapshot versions.

This allows:

Replay plant behavior
using exact logic version

Very powerful for debugging.

Why This Feature Matters

Industrial control systems run for 10–30 years.

Versioning ensures engineers can:

• trace logic history

• audit commissioning changes

• safely roll back bad logic

• reproduce system behavior

This adds enterprise-grade reliability to the platform.

Development Effort

Estimated work:

Git integration: ~150 lines
Dolt database versioning: ~80 lines
UI version panel: ~120 lines
Snapshot manager: ~90 lines

Total:

~400 lines of code.

Result

The platform now supports:

• automated version control

• plant model versioning

• logic diff comparison

• rollback capability

• engineering audit history

Structured Text Engineering Editor

Purpose

Transform the generated PLC code viewer into a full enterprise-grade Structured Text engineering IDE.

The editor allows engineers to:

• inspect generated logic

• edit routines and function blocks

• organize large automation projects

• create sequences and interlocks

• validate syntax in real time

• autocomplete plant tags

• structure code dynamically from the plant model

This ensures the generated logic is trusted, editable, and production-ready for real industrial projects.

The experience should feel like:

Siemens TIA Portal × Rockwell Studio 5000 × VSCode

while maintaining the simplicity of a modern developer interface.

Core Editor Technology

Editor Engine

Monaco Editor

Provides:

• syntax highlighting

• multi-file editing

• inline errors

• search / replace

• code navigation

• diff viewer

• large file support

Structured Text Parsing

Tree-Sitter Structured Text Grammar

Used for:

• syntax validation

• token parsing

• error highlighting

• code folding

Backend Logic Service

FastAPI Code Service

Handles:

• logic saving

• syntax validation

• routine management

• version snapshots

Tag Autocomplete Source

Neo4j Plant Graph

Autocomplete queries read device and signal tags from the plant model.

Example query:

```
MATCH (n)
WHERE n.type IN ["sensor","actuator","pump","valve","motor"]
RETURN n.tag
```

This ensures tag suggestions always match the plant configuration.

Enterprise Logic File Architecture

Hardcoding device names (such as pump_control or tank_level_control) breaks vendor-agnostic architecture.

Instead, the logic structure is generated dynamically from the plant graph, making it compatible with any industrial system.

Universal Logic File Structure

The system organizes logic using control layers and functional categories.

```
/control_logic
  main.st
  sequences/
    sequence_001.st
    sequence_002.st
  equipment/
    device_001.st
    device_002.st
  control_loops/
    loop_001.st
    loop_002.st
  interlocks/
    interlock_001.st
    interlock_002.st
  alarms/
    alarm_group_001.st
  utilities/
    math_functions.st
    timers.st
```

Dynamic Logic Generation

The backend reads the plant knowledge graph and categorizes devices automatically.

Example plant graph nodes:

PUMP101
VALVE201
LT101
FT201

Generated routines:

```
equipment/
  pump101_control.st
  valve201_control.st
```

This generation occurs only after parsing plant metadata.

No device assumptions are hardcoded.

Generic Logic Folder Structure

Every project begins with a universal skeleton.

```
/control_logic
  main.st
  equipment/
  control_loops/
  sequences/
  interlocks/
  alarms/
  utilities/
```

Files inside are generated dynamically.

Example Generated Project

```
/control_logic
  main.st
  equipment/
    pump_101.st
    valve_201.st
  control_loops/
    level_loop_101.st
```

```
    flow_loop_201.st
  sequences/
    startup_sequence.st
    shutdown_sequence.st
  interlocks/
    pump_safety_interlock.st
  alarms/
    tank_alarm_group.st
```

Another project might generate:

```
/control_logic
  main.st
  equipment/
    conveyor_01.st
    robot_arm_02.st
  control_loops/
    speed_control_loop.st
  sequences/
    packaging_sequence.st
```

The architecture works for any industrial environment:

- water treatment
- packaging lines
- robotics cells
- chemical plants
- food processing
- power systems

Main Routine Behavior

The main.st program orchestrates all routines.

Example:

PROGRAM Main

```
Equipment_Pump101();
Equipment_Valve201();

LevelLoop101();
FlowLoop201();

StartupSequence();

AlarmManager();

END_PROGRAM
```

This mirrors professional PLC program structures used in large automation systems.

Routine Explorer (UI)

The left panel of the editor displays a dynamic Routine Explorer.

Example UI structure:

```
CONTROL LOGIC

Main
Equipment
   Pump_101
   Valve_201
Control Loops
   Level_Loop_101
Sequences
   Startup
   Shutdown
Interlocks
   Pump_Safety
Alarms
Utilities
```

Engineers can:

• create routines

• rename routines

• duplicate logic blocks

• reorganize code modules

Interactive Structured Text Editor

Example editable code:

```
PROGRAM Pump_101

IF LT101 > 90 THEN
   PUMP101 := FALSE;
END_IF;

END_PROGRAM
```

Capabilities:

• real-time syntax validation

• autocomplete for plant tags

• bracket matching

• multi-file editing

• inline diagnostics

Tag Autocomplete

Autocomplete suggestions are pulled directly from the plant graph.

Example:

Typing:

LT

Suggestions:

LT101
LT102
LT201

Typing:

PUMP

Suggestions:

PUMP101
PUMP102
PUMP201

This prevents tag errors and accelerates engineering workflows.

Logic Templates

Quick insert menu:

Insert Template

IF / THEN
PID Control Loop
Motor Start Sequence
Pump Interlock
Alarm Block
Safety Shutdown

Templates generate Structured Text skeletons automatically.

Syntax Validation

Structured Text is validated on save.

Example error:

ERROR
Missing END_IF

Line 42

The editor highlights the issue and prevents deployment until resolved.

Version Integration

Every routine is automatically connected to the platform’s versioning and snapshot system.

Engineers can:

• view historical versions

• compare logic revisions

• rollback changes

Example diff:

Version 1.3
IF LT101 > 90 THEN

Version 1.4
IF LT101 > 95 THEN

Performance and Scalability

To support large enterprise projects:

• lazy loading of routines

• virtualized editor rendering

• indexed PostgreSQL storage

• incremental save operations

This allows thousands of routines and hundreds of engineers per system.

Result

The platform now includes a fully enterprise-grade Structured Text development environment that mirrors major industrial IDEs while remaining fully vendor-agnostic.

It provides:

• dynamic plant-driven logic architecture

• professional routine organization

• real engineering editing capabilities

• plant-aware autocomplete

• integrated version control

Automated IO Mapping Engine (Enterprise-Grade)

Purpose

Provide a fully automated, enterprise-grade IO mapping system that generates PLC IO configurations directly from the plant model.

This eliminates the traditional manual workflow where engineers map tags to PLC IO channels using spreadsheets and vendor configuration tools.

The engine must support:

• large industrial plants (thousands of IO points)

• multi-PLC architectures

• automatic tag classification

• automatic slot/channel assignment

• runtime deployment integration

• export for external PLC systems

The feature integrates directly into the automation pipeline:

documents → plant graph → control logic → IO mapping → PLC runtime → simulation → monitoring

This ensures IO configuration is generated automatically before deployment, allowing the PLC runtime to start immediately after logic generation.

Core Tools (All Open Source)

Backend Framework

FastAPI

Handles:

• IO mapping generation requests

• device metadata queries

• mapping export endpoints

• runtime configuration

Data Processing Engine

Python + Pandas

Used for:

• signal classification

• IO grouping

• mapping generation

• large dataset handling

Pandas allows the engine to efficiently process thousands of signals.

Plant Model Source

Neo4j

Provides the device and signal relationships extracted earlier in the workflow.

Example nodes:

Pump
Valve
Sensor
Motor
Flowmeter

Example relationships:

Sensor → Tank

Pump → Pipe
Valve → Tank

These relationships allow the system to determine signal types automatically.

Engineering Metadata Storage

PostgreSQL

Stores the IO mapping tables and configuration state.

Database tables:

io_devices
io_signals
io_channels
io_mappings

PLC Runtime Integration

OpenPLC Runtime API

The IO mapping engine configures the PLC runtime automatically.

Python connects to OpenPLC using its API to assign:

• IO addresses

• input/output types

• tag bindings

System Deployment Environment

All services run inside Docker containers to support enterprise deployments.

Container stack:

FastAPI
Neo4j
PostgreSQL

OpenPLC
Prometheus
Grafana

This mirrors the architecture used in the core automation pipeline.

Automated IO Mapping Workflow

Step 1 — Extract Devices

The engine queries the plant graph.

Example query:

```
MATCH (n)
WHERE n.type IN ["sensor","actuator","pump","valve","motor"]
RETURN n
```

Returned nodes contain metadata such as:

tag
device_type
signal_type
equipment_id

Step 2 — Classify Signals

The engine determines IO type automatically.

Classification rules:

Device Type	IO Type
Level Sensor	Analog Input

Flow Sensor	Analog Input
Temperature Sensor	Analog Input
Motor Start	Digital Output
Valve Command	Digital Output
Motor Current	Analog Input

These rules are configurable in a YAML configuration file.

Example:

io_classification_rules.yaml

Step 3 — Assign IO Channels

The system allocates slots and channels automatically.

Example algorithm:

```
analog_inputs += 1
digital_outputs += 1

slot = device_index // 16
channel = device_index % 16
```

This supports modular PLC racks.

Step 4 — Generate IO Mapping Table

The engine produces a structured mapping dataset.

Example enterprise-grade mapping table:

TAG	DEVICE TYPE	SIGNAL TYPE	IO TYPE	PLC	SLOT	CHANNEL	DESCRIPTION
LT101	Level Sensor	Level	AI	PLC1	3	1	Tank Level
FT201	Flow Sensor	Flow	AI	PLC1	3	2	Aeration Flow
PUMP101	Motor	Start Cmd	DO	PLC1	2	4	Pump Start
VALVE201	Control Valve	Position Cmd	DO	PLC1	2	6	Discharge Valve
MTR101	Motor Current	Current	AI	PLC1	3	5	Pump Motor Current

The table structure is intentionally dense and engineering-grade.

Database Schema

The IO mapping is stored in PostgreSQL.

Example schema:

table io_mapping

id
tag
device_type
signal_type
io_type
plc_id
slot
channel
description
equipment_id
created_at

Indexes are applied to:

tag
equipment_id
plc_id

This allows the system to scale to tens of thousands of signals.

UI Integration

The feature integrates into the Automation Build Studio UI.

The interface uses a 3-panel engineering layout with a left project navigator, central system visualization, and a right engineering panel.

The IO mapping module appears inside the Right Engineering Panel.

Details
Signals
Trace
Replay
IO Mapping
Diagnostics

IO Mapping UI Panel

Example interface:

IO MAPPING TABLE

TAG	TYPE	IO	PLC	SLOT	CHANNEL
LT101	Analog	AI	PLC1	3	1
FT201	Analog	AI	PLC1	3	2
PUMP101	Digital	DO	PLC1	2	4
VALVE201	Digital	DO	PLC1	2	6

Controls:

Generate Mapping
Auto-Assign Channels
Export CSV
Export PLC Tags
Apply to Runtime
Validate Mapping

Large Project Features

To support enterprise environments:

• server-side pagination

• column filtering

• search by tag or equipment

• multi-PLC partitioning

• mapping validation rules

Example validation:

duplicate tag detection
invalid IO type
channel overflow
missing signal

Validation warnings appear in the UI.

Deployment Integration

Before PLC deployment, the system runs:

Generate IO Mapping
↓
Validate Mapping
↓
Apply to PLC Runtime

The OpenPLC runtime then receives the IO configuration automatically.

Example runtime call:

POST /openplc/configure_io

This removes manual configuration entirely.

Export Options

Engineers can export mappings for external tools.

Supported formats:

CSV
JSON
PLC Tag Import
OpenPLC Configuration

Future extensions can include vendor-specific exports.

Integration With Other Platform Features

Plant Graph

Provides the source of device metadata.

Control Logic Generation

IO tags generated during logic creation automatically appear in the mapping table.

Version Control System

Mapping changes are automatically committed to the engineering version system.

This ensures IO configuration changes are fully traceable.

Monitoring

Prometheus metrics reference the same IO tag identifiers, ensuring monitoring dashboards remain synchronized with the IO mapping.

Result

The platform now includes a fully automated industrial IO engineering system that:

• generates IO mapping automatically

• scales to large plants

• integrates with PLC deployment

• supports enterprise traceability

• remains fully open source